

CiT Backend Engineering Track

Week 14: Secure Coding & Common Attacks

The OWASP Top 10, Injection, Access Control & Secrets

Detail	Information
Programme	CiT Backend Engineering Track
Phase	Phase 2 — Specialisation (Weeks 9–20)
Week	Week 14
Instructor	Jim & Anderson
Duration	6 months 24 weeks
Tech Stack	Java 25, Spring Boot 4, Spring Security, PostgreSQL, Postman

Confidential — For CiT Trainees Only

Opening Prayer

We begin, as always, by acknowledging that every gift of understanding comes from God. Please bow your heads as we invite His presence into this session.

Prayer of Dedication

Heavenly Father,

Today we learn to be discerning — to test what comes to us rather than trust it blindly, to guard against what would do harm, and to hold fast to what is good and true. Give us wisdom to see what is hidden, patience to check carefully, and integrity in all that passes through our hands.

As 1 Thessalonians 5:21 instructs us, "Test all things; hold fast what is good." Lord, make that our habit in our work and in our lives — quick to examine, slow to be deceived, and faithful to keep what is right.

May we be diligent students who do not just copy but truly understand. May we encourage one another when we struggle. And may everything we build be offered as service to others.

In Jesus' name we pray,

Amen.

Scripture for the Week

"Test all things; hold fast what is good."

— 1 Thessalonians 5:21 (NKJV)

Table of Contents

1. Welcome to Week 14
2. Thinking Like an Attacker & the OWASP Top 10
3. SQL Injection
4. Cross-Site Scripting (XSS)
5. Broken Access Control
6. Secrets Management
7. Secure Input, Validation & Mass Assignment
8. Security Hygiene — HTTPS, Dependencies & Logging
9. Key Takeaways, Homework & Exercises

1. Welcome to Week 14

Last week you locked the front door: authentication proves who is calling, and authorisation controls what they may do. But a determined attacker does not knock politely at the front door — they test every window and every lock. This week you learn to think like them, so you can defend against them.

Secure coding is not a feature you add at the end; it is a mindset you carry through every line. We will study the most common ways applications are attacked — collected in the industry-standard OWASP Top 10 — and exactly how to defend against each. Reassuringly, you have already been building many of these defences without naming them.

This is career-defining material. A single mistake here can expose thousands of people's money, grades, or identities. Treat it with the weight it carries.

What to Expect in Week 14

By the end of this session, you will be able to:

- Adopt a security mindset — assume every input is hostile
- Name the most critical risks in the OWASP Top 10
- Explain SQL injection and how parameterised queries prevent it
- Explain cross-site scripting (XSS) and how frameworks defend against it
- Recognise broken access control — the most common serious flaw — and prevent it
- Manage secrets safely and never commit them to source control
- Apply secure input handling, avoid mass assignment, and practise good security hygiene

2. Thinking Like an Attacker & the OWASP Top 10

Defence begins with a shift in mindset. A careless developer asks "how do I make this work?" A secure developer also asks "how could someone abuse this?" You must learn to see your own code the way an attacker would.

2.1 The One Rule — Never Trust the Client

Everything that arrives from outside your server is potentially hostile: form fields, URLs, headers, uploaded files — all of it. A friendly user sends a name; an attacker sends carefully crafted text designed to break your assumptions. Treat every input as guilty until validated.

2.2 The OWASP Top 10

OWASP (the Open Worldwide Application Security Project) publishes the Top 10 — a regularly updated list of the most critical web application security risks. It is the industry's shared checklist. We focus this week on the ones you are most likely to meet.

Risk	In Plain English
Broken Access Control	Users can do or see things they should not be allowed to (the #1 risk)
Injection	Hostile input is executed as code — e.g. SQL injection
Cryptographic Failures	Sensitive data is exposed — weak hashing, no HTTPS, leaked secrets
Security Misconfiguration	Insecure defaults left on — open admin endpoints, verbose errors
Vulnerable Components	Using libraries with known security holes

Security Is Everyone's Job

You do not need to be a security specialist to write secure code — but every engineer is responsible for not introducing these flaws. The defences in this session are simple habits. Practised consistently, they prevent the overwhelming majority of real-world breaches.

3. SQL Injection

SQL injection is the classic attack, and still one of the most damaging. It happens when user input is glued directly into a SQL query, letting an attacker rewrite the query itself.

3.1 How the Attack Works

Imagine building a query by joining strings with whatever the user typed:

```
// DANGEROUS — never build SQL like this
String sql = "SELECT * FROM users WHERE name = " + input + "";
```

A normal user types Alice. But an attacker types:

```
' OR '1'='1
```

Now the query becomes WHERE name = " OR '1'='1' — which is always true, so it returns every user in the table. A cleverer payload could delete data or bypass a login entirely.

3.2 The Defence — Never Concatenate; Parameterise

The fix is absolute: never build queries by joining strings. Use parameterised queries, where the input is sent separately from the query and can never change its structure. You already do this correctly:

- JDBC: PreparedStatement with ? placeholders (Week 6)
- Spring Data: derived methods and @Query with :named parameters (Week 11)

```
@Query("SELECT s FROM Student s WHERE s.name = :name")
List<Student> findByName(@Param("name") String name); // SAFE
```

The Rule That Ends SQL Injection

Never place user input into a query by string concatenation — not once, not "just this quick admin script". Always pass input as a parameter. Follow this single rule everywhere and SQL injection simply cannot happen. This is why the framework tools you already use are safe by design.

4. Cross-Site Scripting (XSS)

Where SQL injection attacks your database, cross-site scripting attacks your users. An attacker sneaks malicious JavaScript into content that other people's browsers then run — as if your site itself had written it.

4.1 How the Attack Works

Suppose your app lets users post comments, and you display them raw. An attacker posts a "comment" that is really a script:

```
<script>stealCookies(document.cookie)</script>
```

If your page inserts that comment into the HTML without escaping it, every visitor who views the comment silently runs the attacker's script — which could steal their session, their token, or redirect them to a fake login.

4.2 The Defence — Treat Content as Data, Not Code

The rule is to escape output: when showing user-provided content, render it as plain text, never as live HTML. The good news is that modern frameworks do this for you by default:

- On the frontend, Angular auto-escapes everything in `{{ }}`, and the DOM's `textContent` (Week 6) is always safe
- The danger is deliberately using raw-HTML insertion (`innerHTML` and similar) on untrusted content

Your backend responsibilities: validate and, where appropriate, sanitise input; store content as data; return correct content types; and never reflect raw user input back in a response or an error message.

Two Tracks, One Defence

XSS is exactly why the Frontend track was taught to prefer `textContent` over `innerHTML` back in Week 6, and why Angular escapes by default. Security is a shared responsibility: the backend must not store or reflect hostile content, and the frontend must not render it as code. Both halves guard the same user.

5. Broken Access Control

This is the most common serious flaw in real applications — the OWASP #1 risk — and it is subtle, because the code often looks perfectly fine. It is the gap between authentication and authorisation.

5.1 Authenticated Is Not Authorised

Last week you made endpoints require a valid token, so only logged-in users can call them. But that only proves who the caller is — not that they are allowed to touch this particular thing. Forgetting the second check is broken access control.

5.2 The Classic Example — IDOR

Consider GET /api/v1/students/2024001. Your code checks the caller has a valid token, then returns that student. But what if student 2024001 is not the caller? A logged-in student simply changes the number in the URL to 2024002 and reads someone else's record. This is called an Insecure Direct Object Reference (IDOR), and it is everywhere.

```
// INSECURE: authenticated, but never checks ownership
@GetMapping("/{regNumber}")
public StudentResponse get(@PathVariable String regNumber) {
    return service.findByRegNumber(regNumber); // whose record?
}
```

5.3 The Defence — Check Permission on Every Request, Server-Side

For every protected action, verify not just that the caller is logged in, but that they are permitted to act on this specific resource — checking ownership or role. And do it on the server, always. The user interface hiding a button proves nothing; the attacker calls your API directly.

Hiding a Button Is Not Access Control

The Frontend track can hide a "Delete" button from ordinary users — that is good user experience, but it is not security. Anyone can call your API directly with Postman, ignoring your UI entirely. Every access decision must be enforced again on the server, on every single request. Never rely on the client to protect your data.

6. Secrets Management

A secret is anything that must stay private: database passwords, API keys, the JWT signing key, third-party credentials. Mishandling secrets is one of the most common — and most damaging — mistakes a junior engineer makes.

6.1 Never in Code, Never in Git

The cardinal rule: secrets must never be written into your source code or committed to version control. Once a secret is pushed to a repository, it lives in the history forever — and if that repository is ever public or leaked, the secret is compromised, even if you delete it later.

```
// NEVER do this
String dbPassword = "SuperSecret123";    // hard-coded secret
String jwtKey = "my-signing-key-in-code"; // catastrophic if leaked
```

6.2 The Right Way — Environment Variables

Read secrets from the environment at runtime, as you saw in Week 9. The code references a variable name; the actual value lives outside the codebase, supplied by the server or a secrets manager.

```
# application.yml — the value comes from the environment, not the file
spring:
  datasource:
    password: ${DB_PASSWORD}
  jwt:
    secret: ${JWT_SECRET}
```

Practice	Why
Read secrets from environment variables	Keeps them out of the codebase and history
Add secret files to .gitignore	Prevents accidental commits
Use a secrets manager in production	Central, encrypted, access-controlled storage
Rotate secrets periodically	Limits the damage if one leaks

A Leaked Key Is a Real Incident

Automated bots constantly scan public repositories for committed passwords and keys, often exploiting them within minutes. A single secret pushed to GitHub is not a small mistake — it is a security incident that can cost an organisation dearly. Check every commit; keep secrets out.

7. Secure Input, Validation & Mass Assignment

Two more everyday defences, both of which you have already been building: rigorous input validation, and protecting against mass assignment through DTOs.

7.1 Validation Is a Security Control

The Bean Validation you learned in Week 12 is not only about tidy error messages — it is a frontline defence. Rejecting malformed, over-long, or out-of-range input at the boundary stops many attacks before they begin. Prefer whitelisting (define exactly what is allowed) over blacklisting (trying to list everything forbidden), because attackers are endlessly creative about what to forbid.

7.2 Mass Assignment

Mass assignment is a quiet but serious flaw. If you bind an incoming request body directly onto your entity, an attacker can set fields you never intended to expose — including dangerous ones.

```
// DANGEROUS: binds the whole request onto the entity
@PostMapping
public User create(@RequestBody User user) { ... }
```

If User has a role field, an attacker simply adds "role": "ADMIN" to their JSON and promotes themselves to administrator. The server obediently sets it.

7.3 The Defence — DTOs Again

The request DTOs from Week 10 close this hole completely. A UserRequest DTO contains only the fields a client is allowed to set — a name, an email — and nothing else. There is simply no role field to attack. The DTO is a whitelist of accepted input.

Another Reason DTOs Matter

You first met DTOs as a way to shape clean responses. Here they earn their keep as a security control: by binding only the exact fields you expose, they make mass assignment impossible. Good design and good security are usually the same thing.

8. Security Hygiene — HTTPS, Dependencies & Logging

Beyond specific attacks, secure systems are kept safe by steady, unglamorous habits. These are the ones that matter most in day-to-day work.

8.1 Everyday Habits

Habit	Why It Matters
Always use HTTPS/TLS	Encrypts data in transit so it cannot be read or altered on the wire
Keep dependencies updated	Known vulnerabilities in old libraries are actively exploited
Never log secrets or tokens	Logs are widely readable; a logged password is a leaked password
Rate-limit sensitive endpoints	Slows brute-force attacks on login and reduces abuse
Apply least privilege	Give each account and service only the access it truly needs
Return safe error messages	Leak no stack traces or internals to clients (Week 12)

8.2 Vulnerable Dependencies Are Real

Most of your application is code you did not write — the libraries you depend on. When a serious hole is found in a popular library (as happened with the widely used Log4Shell vulnerability), every app using the old version is exposed overnight. Keep dependencies current, and pay attention to security advisories.

Exercise: Security-Audit Your Library API

Review the Library Management System you have been building against this week:

- Confirm no query is built by string concatenation, and no secret is committed to git
- Add an ownership/role check to any endpoint that returns or changes a specific record (fix IDOR)
- Verify every write endpoint accepts a request DTO, not an entity (no mass assignment)

- Confirm errors return safe messages and that no password or token is ever logged

9. Key Takeaways, Homework & Exercises

9.1 What You Learned This Week

- The security mindset — never trust the client — and the OWASP Top 10 risks
- SQL injection and the absolute rule: parameterise, never concatenate
- Cross-site scripting (XSS) and escaping output — a shared frontend/backend defence
- Broken access control and IDOR — enforce authorisation on every request, server-side
- Secrets management — never in code or git; read from the environment
- Validation as a security control, and DTOs preventing mass assignment
- Security hygiene — HTTPS, updated dependencies, safe logging, rate limiting, least privilege

9.2 Homework

1. Complete the security audit of your Library Management System and push the fixes to GitHub
2. Write down, for each OWASP risk covered this week, the specific defence you applied in your project
3. Move every secret in your project to environment variables and confirm none remain in git history
4. Preview Week 15: Read what OAuth2 and OIDC are, and what Keycloak does. Come with one question ready.

Coming Next — Week 15: OAuth2, OIDC & Keycloak

Jim & Anderson will complete the security module with delegated identity: OAuth2 and OpenID Connect (OIDC), and how Keycloak — the identity platform in the XENO stack — handles single sign-on, tokens, and user management for you. You will then finish securing the module project end to end.

Closing Reflection

"Be sober, be vigilant; because your adversary the devil walks about like a roaring lion, seeking whom he may devour."

— 1 Peter 5:8 (NKJV)

Security is vigilance made practical. There are real adversaries who would exploit any carelessness to harm the people who trust your systems. Stay sober and

watchful — in your code and in your life — for the one who guards well protects not only data, but people.

Glossary of Terms

Term	Definition
Broken Access Control	Users being able to do or see things they are not permitted to — the top OWASP risk
IDOR	Insecure Direct Object Reference — changing an id in a URL to access another user's data
Injection	An attack where hostile input is executed as code, e.g. SQL injection
Least Privilege	Granting each account or service only the access it truly needs
Mass Assignment	Binding a request directly to an entity, letting attackers set unintended fields
OWASP Top 10	The industry-standard list of the most critical web application security risks
Parameterised Query	A query where input is sent separately and cannot alter the query structure
Rate Limiting	Capping how often an endpoint can be called, to slow brute-force and abuse
Sanitisation	Cleaning input/output of dangerous content before storing or rendering it
Secret	A private value (password, key, token) that must never appear in code or git
SQL Injection	Injecting SQL through unsanitised input glued into a query
TLS / HTTPS	Encryption of data in transit so it cannot be read or altered on the wire
XSS	Cross-Site Scripting — injecting scripts that run in other users' browsers